Complexitylib:
An index of proved statements

19 October 2026

This tour collects 75 entries spanning the library's significant results, from machine complexity and descriptive logic to cryptography, circuit lower bounds, proof complexity, communication, coding and circuit synthesis. Some entries group closely related theorems. It is a statement index, not an exhaustive API reference or a translation of proof bodies. Each entry identifies its hypotheses, model, quantitative guarantee and scope, with links pinned to reviewed library revisions `8d38f775`, `811ad02a` (first-order projections), and `4f6bdaf3` (basic class developments), and `2f492c03` (small counting classes), `814e76e2` (wide gaps and the Boolean hierarchy), `e903aba8` (approximate counting), and `c7feb301` (counting closure), and `aa72232a` (Tree Evaluation). Selection and wording remain open to Samuel's editorial direction.

Unless specified otherwise, encoded decision problems use an injective encoding into finite bit strings, and input size is encoded length. Class statements and reductions refer to the library's machine definitions. Circuit-family results are nonuniform unless uniformity is explicitly asserted. A circuit-size bound does not by itself certify a constructor's runtime; a decoder's correctness does not by itself certify its running time. Assumptions such as one-way functions or circuit hardness are premises of implications, not established existence results.

Source citations credit the mathematics; linked modules record formalization authors and port provenance. This index was prepared and independently reviewed with AI assistance. Lean checks the formal declarations; the prose-to-Lean comparison is a separate review, not a machine-checked equivalence proof.

Contents

1 Completeness and approximation **2**
2 Logic, diagonalization and the polynomial hierarchy **4**
3 Space, alternation and interaction **5**
4 Randomness and counting **7**
5 Cryptographic constructions **13**
6 Circuit depth, lower bounds and barriers **15**
7 Proof complexity **18**
8 Communication and formulas **18**
9 Boolean analysis and expander walks **19**
10 Sensitivity and query complexity **20**
11 Finite combinatorial inequalities **21**
12 Reed–Solomon decoding **22**
13 Arithmetic circuits and simultaneous computation **22**
14 Circuits for iterated functions **23**

1 Completeness and approximation

1.1 Cook–Levin and its complement

SAT is NP-complete and UNSAT is coNP-complete under tidy polynomial-time many-one reductions, using the library's encoding `encCNF` of CNF formulas. Reduction efficiency is proved, not assumed. SAT is also NP-complete when every clause has exactly three literal occurrences: repeated literals are permitted. This does not establish the variant requiring three distinct literals per clause.[6, Section 2.3]

`Cslib.Complexity.CookLevin.SAT_npComplete`
`Cslib.Complexity.CookLevin.UNSAT_coNPComplete`
`Cslib.Complexity.ExactThreeSAT_npComplete`

1.2 DP and SAT–UNSAT completeness

The difference class DP consists of languages $B \cap C$ with $B \in \mathsf{NP}$ and $C \in \mathsf{coNP}$, equivalently differences of two NP languages. For any NP-complete B and coNP-complete C, the product language $B \times C$ is DP-complete under polynomial-time many-one reductions, using the product encoding. In particular, pairs of CNF formulas (φ, ψ) with φ satisfiable and ψ unsatisfiable form a DP-complete problem. The theorem quantifies over all injectively encoded source types; the two component reductions and their pairing have certified polynomial time. The definition permits distinct NP and coNP components; it does not require the resulting language to belong to both classes.[71]

`Cslib.Complexity.NPComplete.prod_coNPComplete`
`Cslib.Complexity.SATUNSAT_dpComplete`

The complement class coDP consists of unions of an NP language and a coNP language; it is closed under union and polynomial-time many-one reductions.

`Cslib.Complexity.mem_coDP_iff_union`
`Cslib.Complexity.union_mem_coDP`
`Cslib.Complexity.PolyTimeReducible.mem_coDP`

1.3 The Boolean hierarchy over NP

Using the difference-hierarchy convention,

$$\mathsf{BH}_0 = \{\varnothing\}, \qquad \mathsf{BH}_{n+1} = \{B \setminus D : B \in \mathsf{NP},\ D \in \mathsf{BH}_n\}.$$

We prove $\mathsf{BH}_1 = \mathsf{NP}$ and $\mathsf{BH}_2 = \mathsf{DP}$, monotonicity of levels, complement membership one level higher, and polynomial-time many-one reduction closure at each fixed level. Consequently the union of the finite levels is closed under complement. The construction and inverse-image theorem are generic over the base class of sets; NP supplies the complexity instance. This follows the difference presentation of the Boolean hierarchy; no separation between levels or hierarchy-collapse theorem is asserted.[19]

`Cslib.Complexity.BH_one`
`Cslib.Complexity.BH_two`
`Cslib.Complexity.BH_mono`
`Cslib.Complexity.compl_mem_BH_succ`
`Cslib.Complexity.compl_mem_BooleanHierarchy`
`Cslib.Complexity.PolyTimeReducible.mem_BH`

1.4 Graph and number problems are NP-complete

Three-colorability, undirected Hamiltonian cycle, binary subset sum, and the budget decision version of traveling salesman are NP-complete under certified polynomial-time many-one reductions. Subset sum selects entries of a list of natural numbers, allowing repeated values; its binary encoding does not give strong NP-hardness. TSP uses binary natural distances and budget, and does not require symmetry or the triangle inequality.[33, 52, 82]

`Cslib.Complexity.ThreeColoring_npComplete`
`Cslib.Complexity.HamCycle_npComplete`
`Cslib.Complexity.SubsetSum_npComplete`
`Cslib.Complexity.TSP_npComplete`

Circuit evaluation. Given a one-output De Morgan circuit and an input of the matching arity, deciding whether its output is true is in P. Instances are well formed by their type; `encCircuitValue` encodes the circuit and input bits. The standard P-completeness theorem under logspace reductions is not yet formalized here. [59]

`Cslib.Complexity.CircuitValue.mem_P`

1.5 The PCP theorem

$$\mathsf{NP} = \mathsf{PCP}(O(\log n), O(1)).$$

The verifier is polynomial-time and nonadaptive, has perfect completeness and soundness at most $1/2$, and uses logarithmically many random bits and constantly many queries. In the Lean equality the bound functions are $\lfloor \log_2 n \rfloor + 1$ (with Lean's natural-number logarithm) and 1; the class definition allows constant multiples of both. It does not assert one query.[25]

`Cslib.Complexity.NP_eq_PCP_log`

1.6 A constant gap for MAX-3SAT

It is NP-hard, under polynomial-time promise reductions, to distinguish a satisfiable formula with clauses of at most three literals from one in which every assignment satisfies at most the fraction

$$1 - \frac{1}{3 \cdot 2^{77}}$$

of its clauses. Clauses are counted with multiplicity. This is the explicit gap obtained from the formalized PCP construction; no optimal approximation threshold is asserted.[7, 25]

`Cslib.Complexity.GapThreeSAT_npHard`

1.7 PCP without random bits

For the same nonadaptive polynomial-time verifier model,

$$\mathsf{PCP}(0,0) = \mathsf{P}, \qquad A \in \mathsf{NP} \iff \exists k \in \mathbb{N},\ A \in \mathsf{PCP}\big(0, n \mapsto (n+1)^k\big).$$

The class definition allows constant multiples of the query bound. With no random bits, perfect completeness and soundness at most $1/2$ force deterministic acceptance of an appropriate proof for a member and rejection of every proof for a nonmember. These identities hold for each injective bit-string encoding. [6, Chapter 11]

`Cslib.Complexity.PCP_zero_zero`
`Cslib.Complexity.mem_NP_iff_exists_PCP_zero`

2 Logic, diagonalization and the polynomial hierarchy

2.1 Fagin's theorem

For a finite relational vocabulary, a class of finite structures is definable by an existential second-order sentence exactly when it is closed under isomorphism and its encoded query language belongs to NP. The encoding gives universe size in unary and framed relation truth tables; malformed tables are rejected. Both directions connect the logical semantics to the machine-backed verifier class; structures of cardinality zero and one are included.[27, 46]

`Cslib.Logic.ESO.definable_iff_isoClosed_and_queryLanguage_mem_NP`

First-order projection infrastructure. With separate numeric and input vocabularies, each projection formula selects a constant or one signed input-relation bit using numeric tests alone. For a fixed numeric structure and variable assignment, the same selected literal works for every interpretation of the input relations. The finite numeric cases are mutually exclusive and exhaustive. Projections with finite coordinate types compose, with the composite structure isomorphic to successive interpretation; quantifier-free numeric definitions and guards remain quantifier-free under composition. A concrete client is loop-free graph complement. [47, Definitions 3.3–3.6]

This is a vocabulary-parametric, relativized framework: output domains may be restricted by numeric formulas, and interpreted vocabularies are relational. It does not yet specialize to positive-dimensional full tuple universes with canonical order and BIT, certify a logspace transducer, or prove Circuit Value completeness. The classical quantifier-free convention of the cited source excludes BIT.

`Cslib.Logic.FO.ProjectionFormula.exists_literal`
`Cslib.Logic.FO.Projection.compEquiv`
`Cslib.Logic.FO.Projection.IsQF.comp`

2.2 Polynomial time is strictly smaller than exponential time

For languages of finite bit strings,

$$\mathsf{P} \subsetneq \mathsf{EXP}.$$

The separating language is given by a machine diagonalization. Here EXP is the union of deterministic time classes with bounds 2^{n^k}. The supporting time hierarchy uses a polynomial simulation gap; this entry does not claim the sharp $f \log f$ hierarchy.[6, Chapter 3]

`Cslib.Complexity.P_ssubset_EXP`

2.3 Ladner's theorem

If $\mathsf{P} \neq \mathsf{NP}$, there exists a language of bit strings in NP which is neither in P nor NP-complete under polynomial-time many-one reductions. The witness is a padded satisfiability language with a polynomial-time padding exponent. This is a conditional intermediate-degree theorem; it does not assume an intermediate problem as additional input.[6, Theorem 3.3]

`Cslib.Complexity.Ladner.exists_NP_not_P_not_NPComplete`

2.4 A Baker–Gill–Solovay oracle separation

There exists an oracle language A such that

$$\mathsf{P}^A \subsetneq \mathsf{NP}^A.$$

The classes use the library's certified polynomial-time oracle programs and oracle verifiers. This is the separating-oracle half of the relativization phenomenon, rather than a claim here of both a separating and a collapsing oracle.[8]

`Cslib.Complexity.exists_PWithOracle_ssubset_NPWithOracle`

2.5 Karp–Lipton collapse

If SAT has polynomial-size Boolean circuits on its encoded inputs, then

$$\mathsf{PH} = \Sigma_2^p.$$

In particular this follows if every NP problem, under its encoding, is the preimage of a language in P/poly. The polynomial-time computation of the self-reduction and evaluation of circuit advice are part of the development. The circuit hypothesis is nonuniform.[53]

`Cslib.Complexity.PH_eq_SigmaP_two_of_SAT`
`Cslib.Complexity.PH_eq_SigmaP_two_of_NP_subset_PPoly`

2.6 Plain Kolmogorov complexity is uncomputable

Let $K(x)$ be the minimum bit-string description length of $x \in \mathbb{N}$ under the library's universal partial-computable description method. Then $K(x) \to \infty$ as $x \to \infty$, but every total computable unbounded function $f : \mathbb{N} \to \mathbb{N}$ satisfies $K(x) < f(x)$ for infinitely many x. In particular, K is not computable. This is plain complexity, often denoted C in the literature, not prefix-free complexity.[61]

`Cslib.Kolmogorov.DescriptionMethod.tendsto_complexity_universal`
`Cslib.Kolmogorov.DescriptionMethod.frequently_complexity_universal_lt_of_computable`
`Cslib.Kolmogorov.DescriptionMethod.not_computable_complexity_universal`

3 Space, alternation and interaction

Containment map. The proved machine-class relationships include

$$\mathsf{NL} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PH} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}, \qquad \mathsf{AP} = \mathsf{NPSPACE} = \mathsf{PSPACE}.$$

The equalities and the NL inclusion receive entries below. The links here supply the remaining arrows, including PH containment in polynomial space and the deterministic exponential-time simulation of polynomial space. Inclusion arrows do not assert strictness; the separate $\mathsf{P} \subsetneq \mathsf{EXP}$ theorem appears in the preceding section. [6, 82]

`Cslib.Complexity.P_subset_NP`
`Cslib.Complexity.NP_subset_PH`
`Cslib.Complexity.PH_subset_PSPACE`
`Cslib.Complexity.PSPACE_subset_EXP`

3.1 Nondeterministic logspace lies in polynomial time

$$\mathsf{NL} \subseteq \mathsf{P}.$$

NL is defined by native nondeterministic machines with logarithmic space bounded on every computation prefix and acceptance by an accepting path. A polynomial-time decider exists even though the nondeterministic machine's other branches need not halt. This entry does not assert the still-outstanding native $\mathsf{NL} = \mathsf{coNL}$ theorem.[82, Section 8.5]

`Cslib.Complexity.NL_subset_P`

3.2 Savitch's theorem for polynomial space

$$\mathsf{NPSPACE} = \mathsf{PSPACE}.$$

Both classes are machine-defined. Nondeterministic acceptance is by an accepting path, with every path respecting the space bound; the deterministic decider is total. This is the equality of polynomial-space classes, not a statement of the fully parameterized $\mathsf{NSPACE}(s) \subseteq \mathsf{DSPACE}(s^2)$ theorem. [82, Corollary 8.6]

`Cslib.Complexity.NPSPACE_eq_PSPACE`

3.3 TQBF is PSPACE-complete

The true closed quantified Boolean formulas are PSPACE-complete under polynomial-time many-one reductions. Membership uses an explicit-stack evaluator; hardness includes a certified polynomial-time construction of the quantified reachability formula. Formulas with free variables are excluded from TQBF. [82, Theorem 8.9]

`Cslib.Complexity.TQBF_pspaceComplete`

3.4 Alternating polynomial time equals polynomial space

$$\mathsf{AP} = \mathsf{PSPACE}.$$

AP uses the library's native alternating machines and a polynomial time bound. The equality is with the same machine-defined PSPACE used above; it is not merely an equality of abstract game predicates.[20]

`Cslib.Complexity.AP_eq_PSPACE`

3.5 Interactive proofs lie in polynomial space

$$\mathsf{IP} \subseteq \mathsf{PSPACE}.$$

IP uses a private-coin verifier realized by one fair-coin oracle machine and a polynomial clock valid against every stateful prover. Provers may be randomized and computationally unbounded; completeness is at least $2/3$ and soundness at most $1/3$. The converse $\mathsf{PSPACE} \subseteq \mathsf{IP}$ is not yet formalized here. [6, Chapter 8]

`Cslib.Complexity.IP_subset_PSPACE`

3.6 Graph non-isomorphism has a private-coin interactive proof

Graph non-isomorphism belongs to IP. The strict polynomial-time fair-coin verifier has perfect completeness and soundness at most $(17/32)^2 < 1/3$ after two rounds. It samples relabelings by sorting random vertex keys; this avoids assuming exact uniform sampling of a permutation from a fixed number of fair bits. The result is for private coins and does not establish the public-coin AM protocol. [37]

`Cslib.Complexity.GraphNonIsomorphism.isInteractiveProof_verifier`
`Cslib.Complexity.GraphNonIsomorphism.mem_IP`

3.7 Cook–Mertz Tree Evaluation in small space

A valid instance is a perfect binary tree with every leaf at depth H, with ℓ-bit values at its leaves and a gate $\{0,1\}^\ell \times \{0,1\}^\ell \to \{0,1\}^\ell$ at each internal node. The injective encoding gives ℓ, H in unary, followed by gate truth tables and leaf values in preorder. Write n for its bit length and $b(k)$ for the length of the library's binary encoding of k. One fixed deterministic multi-tape Turing machine computes the root value on every encoded instance, using work space

$$O\big((\ell + H + 1)(b(\ell) + 1) + b(n + 1)\big),$$

that is, $O((\ell + H + 1)\log(\ell + 2) + \log(n + 2))$. The theorem allows zero value length and zero height. Input addressing costs are included, the output is streamed, and a running-time bound exists without an asserted asymptotic rate.

Running once over every nonzero field element, the algebraic procedure preserves the other two registers while adding the scaled tree value to its target register over any finite field with $2\ell < |K| - 1$. The concrete machine chooses a suitable prime, reads the encoded gates, and satisfies a proved state-size invariant. This formalizes the Cook–Mertz construction with Goldreich's value-length parametrization and exposition; the Lean development is by Christian Reitwiessner with disclosed AI assistance.[23, 34] Correctness and resource bounds apply to valid encoded instances; malformed-input rejection is not proved. A separate $O(\log n \log\log n)$ corollary and the refined $O(\ell + H\log(\ell+2))$ parameter bound are not asserted here.

`Cslib.TreeEval.computableTidilyInSpace_eval`
`Cslib.TreeEval.encode_injective`
`Cslib.TreeEval.proc_eq`

4 Randomness and counting

4.1 Sipser–Gács–Lautemann

$$\mathsf{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p.$$

For the library's bounded-error polynomial-time class, membership has both an existential–universal and a universal–existential characterization with polynomially bounded bit-string witnesses and a polynomial-time predicate. [60, 81]

`Cslib.Complexity.BPP_subset_SigmaP_two_inter_PiP_two`

As a consequence of the hierarchy collapse, if $\mathsf{NP} \subseteq \mathsf{P}$ for every type and injective bit-string encoding, then $\mathsf{BPP} = \mathsf{P}$ for every such encoded problem class. The formal premise quantifies over all encodings, rather than assuming equality only at one chosen encoding.[6]

`Cslib.Complexity.BPP_eq_P_of_NP_subset_P`

4.2 Toda's theorem

$$\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}.$$

Here $\#\mathsf{P}$ counts witnesses of polynomial-time verifiers. The target class uses a polynomial-time oracle program answering decision problems via bits of one such counting function. The program is fixed before the oracle is chosen and must work for every oracle language agreeing on valid encoded queries; off-code answers are arbitrary. The target class has no separate query-count parameter. The nondeterministic-machine characterization of $\#\mathsf{P}$ remains separate unfinished work.[86]

`Cslib.Complexity.PH_subset_PWithSharpPOracle`

4.3 Adleman's theorem

$$\mathsf{BPP} \subseteq \mathsf{P/poly}.$$

Every bounded-error polynomial-time language has a nonuniform family of polynomial-size De Morgan circuits. The result also covers encoded problems by taking the preimage of a bit-string language. It does not provide a uniform algorithm to choose the successful random strings or construct the circuits. [2, 6]

`Cslib.Circuits.Boolean.PPoly.exists_of_mem_BPP`

4.4 Derandomization from average-case hardness

Suppose a language L belongs to $\mathsf{E} = \mathsf{DTIME}(2^{O(n)})$ and there is a positive integer c such that, for every sufficiently large n, every De Morgan circuit of size at most $s_n = 2^{\lfloor n/c \rfloor}$ agrees with L on at most $1/2 + 1/s_n$ of the n-bit inputs. Then $\mathsf{P} = \mathsf{BPP}$, uniformly for encoded problems. The Nisan–Wigderson generator and exhaustive seed search have certified polynomial time. Worst-case exponential circuit hardness alone is not the hypothesis proved sufficient here.[68]

`Cslib.Complexity.P_eq_BPP_of_isAverageCaseHard`

4.5 Counting algebra and the GapP characterization of PP

For each fixed encoding, #P is closed under pointwise addition and multiplication, and GapP under addition, subtraction and multiplication; the constant natural-valued and integer-valued functions belong to the respective classes. Both classes are also closed under precomposition by a polynomial-time function between encoded instance types. The resulting count is exactly the original count on the mapped instance; no padding factor is introduced. Moreover,

$$A \in \mathsf{PP} \iff \exists g \in \mathsf{GapP}\ \forall x,\quad x \in A \iff g(x) > 0.$$

Here #P counts accepting certificates of a deterministic polynomial-time verifier of polynomial certificate length; GapP consists of differences of two such counts. These are verifier characterizations. The corresponding nondeterministic-machine characterizations remain unfinished. [29, 88]

`Cslib.Complexity.SharpP.subsemiring`
`Cslib.Complexity.GapP.subring`
`Cslib.Complexity.mem_PP_iff_exists_GapP`
`Cslib.Complexity.SharpP.comp_mem`
`Cslib.Complexity.GapP.comp_mem`

Polynomial-time functions with binary natural output belong to #P; those with binary integer output belong to GapP. The bound is on output bit length, so the numeric values may be exponential in input length. [31, Theorem 3.5]

`Cslib.Complexity.SharpP.of_polyTime`
`Cslib.Complexity.GapP.of_polyTime`

4.6 Boolean closure and unambiguous polynomial time

For every fixed injective encoding, P, NP and coNP are closed under union and intersection. The odd-acceptance class $\oplus$P is closed under union, intersection and symmetric difference. The verifier class UP satisfies

$$\mathsf{P} \subseteq \mathsf{UP} \subseteq \mathsf{NP} \cap \oplus\mathsf{P}.$$

Its verifier has at most one accepting certificate of the specified polynomial length on every input, and exactly one on members. Equivalently, a #P function bounded by one is one precisely on the language. UP is closed under polynomial-time many-one reductions. This establishes the verifier formulation; an equivalence with unambiguous nondeterministic machines is not asserted.[6, 71] The complement class coUP contains P and is contained in both coNP and $\oplus$P; it also inherits polynomial-time reduction closure. These are elementary consequences of the complement convention and the corresponding UP containments.

`Cslib.Complexity.P_subset_coUP`
`Cslib.Complexity.coUP_subset_coNP`
`Cslib.Complexity.coUP_subset_ParityP`
`Cslib.Complexity.inter_mem_P`
`Cslib.Complexity.union_mem_NP`
`Cslib.Complexity.inter_mem_coNP`
`Cslib.Complexity.symmDiff_mem_ParityP`
`Cslib.Complexity.P_subset_UP`
`Cslib.Complexity.UP_subset_NP`
`Cslib.Complexity.UP_subset_ParityP`
`Cslib.Complexity.PolyTimeReducible.mem_UP`

The unique-solution class US instead asks that the accepting count equal one exactly on members; no-instances may have zero or multiple witnesses. We prove $\mathsf{UP} \cup \mathsf{coNP} \subseteq \mathsf{US} \subseteq \mathsf{C}_{=}\mathsf{P}$, intersection closure, and polynomial-time many-one reduction closure. The definition follows Blass–Gurevich; these containments are elementary count identities. Unique-SAT completeness is not asserted here.[14]

`Cslib.Complexity.UP_subset_US`
`Cslib.Complexity.coNP_subset_US`
`Cslib.Complexity.US_subset_CeqP`
`Cslib.Complexity.inter_mem_US`
`Cslib.Complexity.PolyTimeReducible.mem_US`

4.7 Exact counting and PP

For the same verifier-based counting functions, the class $\mathsf{C_{=}P}$ is characterized by

$$A \in \mathsf{C_{=}P} \iff \exists g \in \mathsf{GapP}\ \forall x,\quad x \in A \iff g(x) = 0.$$

Equivalently, membership is equality of two #P functions. This class is closed under union, intersection and polynomial-time many-one reductions, and satisfies

$$\mathsf{coNP} \subseteq \mathsf{C_{=}P} \subseteq \mathsf{PP}.$$

The complement class $\mathsf{coC_{=}P}$ consists of nonzero sets of GapP functions and satisfies $\mathsf{NP} \subseteq \mathsf{coC_{=}P} \subseteq \mathsf{PP}$. It inherits union, intersection and reduction closure. This does not assert $\mathsf{C_{=}P} = \mathsf{coC_{=}P}$ or a counting-machine characterization. [29][31, Classifications 4.5, 4.8–4.9 and Corollaries 4.10–4.11]

`Cslib.Complexity.mem_coCeqP_iff_exists_GapP`
`Cslib.Complexity.NP_subset_coCeqP`
`Cslib.Complexity.coCeqP_subset_PP`
`Cslib.Complexity.mem_CeqP_iff_exists_SharpP`
`Cslib.Complexity.inter_mem_CeqP`
`Cslib.Complexity.coNP_subset_CeqP`
`Cslib.Complexity.CeqP_subset_PP`
`Cslib.Complexity.PolyTimeReducible.mem_CeqP`

4.8 Bounded ambiguity and small gaps

For each injective input encoding, FewP consists of languages recognized by positive #P counts bounded on every input by a polynomial in encoded input length. The certificate-length polynomial and ambiguity-bound polynomial are separate. SPP consists of languages whose characteristic function is in GapP: its gap is exactly one on members and zero on nonmembers. Then

$$\mathsf{UP} \subseteq \mathsf{FewP} \subseteq \mathsf{NP}, \qquad \mathsf{UP} \cup \mathsf{coUP} \subseteq \mathsf{SPP} \subseteq \mathsf{C_{=}P} \cap \mathsf{coC_{=}P}.$$

FewP is closed under union, intersection and polynomial-time many-one reductions; SPP has these closures and complement closure. The elementary proofs reuse the count algebra: FewP union adds ambiguity bounds and intersection multiplies them; SPP complement subtracts its gap from one and intersection multiplies gaps. These are verifier-based results; neither a machine characterization nor the stronger FewP–SPP containment is established by this entry. The FewP definition follows Allender–Rubinstein and the explicit acceptance-set formulation of Borchert–Hemaspaandra–Rothe; the SPP definition and UP inclusion follow Fenner–Fortnow–Kurtz and Fortnow's survey. [5][17, Section 1] [29][31, Classification 4.4 and Corollary 4.11]

`Cslib.Complexity.UP_subset_FewP`
`Cslib.Complexity.FewP_subset_NP`
`Cslib.Complexity.PolyTimeReducible.mem_FewP`
`Cslib.Complexity.UP_subset_SPP`
`Cslib.Complexity.coUP_subset_SPP`
`Cslib.Complexity.compl_mem_SPP`
`Cslib.Complexity.SPP_subset_CeqP`
`Cslib.Complexity.SPP_subset_coCeqP`

4.9 Modular counting

For every fixed natural modulus $k \geq 2$, $\mathsf{Mod}_k\mathsf{P}$ consists of languages with a $\#\mathsf{P}$ witness count f such that

$$x \in A \quad \iff \quad f(x) \not\equiv 0 \pmod{k}.$$

The modulus is a class parameter, not an input. We prove $\mathsf{P} \subseteq \mathsf{UP} \subseteq \mathsf{Mod}_k\mathsf{P}$, $\mathsf{Mod}_2\mathsf{P} = \oplus\mathsf{P}$, and closure under polynomial-time many-one reductions. For prime k, the class is also closed under complement, union and intersection. The complement proof applies Fermat's little theorem to the count $1 + (k-1)f(x)^{k-1}$; intersection uses multiplication and primality. No corresponding composite-modulus Boolean closure is asserted. The total Lean definition also covers the degenerate moduli: at zero it equals NP, and at one only the empty language belongs. The standard $k \geq 2$ convention and nonzero-residue acceptance follow Fortnow's Classification 4.7; the closure proofs are elementary algebraic consequences of the existing counting APIs. [31, Classification 4.7][72]

`Cslib.Complexity.UP_subset_ModP`
`Cslib.Complexity.ModP_two`
`Cslib.Complexity.PolyTimeReducible.mem_ModP`
`Cslib.Complexity.compl_mem_ModP`
`Cslib.Complexity.inter_mem_ModP`
`Cslib.Complexity.union_mem_ModP`

The complement class $\mathsf{coMod}_k\mathsf{P}$ uses zero residues of a $\#\mathsf{P}$ count. It is closed under polynomial-time many-one reductions for every modulus, and equals $\mathsf{Mod}_k\mathsf{P}$ for prime k.

`Cslib.Complexity.mem_coModP_iff_exists_SharpP`
`Cslib.Complexity.PolyTimeReducible.mem_coModP`
`Cslib.Complexity.coModP_eq_ModP`

4.10 Wide counting gaps

For every fixed injective encoding,

$$\mathsf{SPP} \subseteq \mathsf{LWPP} \subseteq \mathsf{WPP} \subseteq \mathsf{C}_{=}\mathsf{P} \cap \mathsf{coC}_{=}\mathsf{P}.$$

In WPP there are a GapP function g and a polynomial-time, binary-integer function f, nonzero on every input, with $g(x) = f(x)$ on members and $g(x) = 0$ on nonmembers. LWPP restricts the target to $f(|x|)$, with f computable from unary length and nonzero on every natural argument, including zero. Negative and exponentially large targets are allowed. Both classes are closed under complement, union and intersection; WPP also has polynomial-time many-one reduction closure. The proofs use exact-gap algebra and the FP-to-GapP bridge. These verifier-based statements follow the definitions of Fenner–Fortnow–Kurtz, as restated by Hemaspaandra et al.; they do not establish counting-machine characterizations, oracle lowness, or robustness under multiple target values. [29][41, Definitions 2.4–2.5]

`Cslib.Complexity.SPP_subset_LWPP`
`Cslib.Complexity.LWPP_subset_WPP`
`Cslib.Complexity.WPP_subset_CeqP`
`Cslib.Complexity.WPP_subset_coCeqP`
`Cslib.Complexity.compl_mem_WPP`
`Cslib.Complexity.inter_mem_WPP`
`Cslib.Complexity.union_mem_WPP`
`Cslib.Complexity.compl_mem_LWPP`
`Cslib.Complexity.inter_mem_LWPP`
`Cslib.Complexity.union_mem_LWPP`
`Cslib.Complexity.PolyTimeReducible.mem_WPP`

4.11 Small bounded-error and almost-wide counting

For every fixed injective encoding,

$$\mathsf{NP} \cup \mathsf{BPP} \subseteq \mathsf{SBP} \subseteq \mathsf{PP}, \qquad \mathsf{SPP} \cup \mathsf{BPP} \subseteq \mathsf{AWPP} \subseteq \mathsf{PP}.$$

SBP has a verifier-based #P function f and a polynomial-time function k with binary natural output such that $f(x) \geq k(x)$ on members and $2f(x) < k(x)$ on nonmembers. The nonnegative threshold may be zero on members; requiring it to be positive on every input gives the same class. AWPP has a verifier-based GapP function g and $p \in \mathbb{N}[X]$ with $0 \leq g(x) \leq 2^{p(|x|)}$ on every input, and

$$x \in A \Rightarrow 3g(x) \geq 2 \cdot 2^{p(|x|)}, \qquad x \notin A \Rightarrow 3g(x) \leq 2^{p(|x|)}.$$

Both classes are closed under polynomial-time many-one reductions between injectively encoded types. AWPP is closed under complement; SBP is closed under union. The AWPP reduction proof permits an input-dependent, unary polynomial-time exponent and pads it to a polynomial in encoded input length. These are elementary consequences of the existing counting algebra and BPP amplification. The definitions follow Watson's formulation of the class introduced by Böhler–Glaßer–Meister, and Fenner's constant-error characterization of AWPP.[15] [90, Definition 2][28] Counting-machine characterizations, $\mathsf{MA} \subseteq \mathsf{SBP} \subseteq \mathsf{AM}$, Uniform AWPP amplification to input-dependent error and PP-lowness remain outstanding. The following entry adds fixed amplification and Boolean closure.

`Cslib.Complexity.NP_subset_SBP`
`Cslib.Complexity.BPP_subset_SBP`
`Cslib.Complexity.SBP_subset_PP`
`Cslib.Complexity.mem_SBP_iff_exists_pos_threshold`
`Cslib.Complexity.union_mem_SBP`
`Cslib.Complexity.PolyTimeReducible.mem_SBP`
`Cslib.Complexity.BPP_subset_AWPP`
`Cslib.Complexity.SPP_subset_AWPP`
`Cslib.Complexity.AWPP_subset_PP`
`Cslib.Complexity.compl_mem_AWPP`
`Cslib.Complexity.mem_AWPP_of_polyTime_exponent`
`Cslib.Complexity.PolyTimeReducible.mem_AWPP`

4.12 Wide gaps, amplification and Boolean closure

For every fixed injective encoding,

$$\mathsf{LWPP} \subseteq \mathsf{WPP} \subseteq \mathsf{AWPP}.$$

The WPP target may be signed, input-dependent and exponentially large: it is a nonzero, binary-output polynomial-time integer function, with the GapP witness equal to that target on members and zero on nonmembers. The classical containment is credited to Fenner–Fortnow–Kurtz–Li; this formalization squares the target and uses binary length to obtain a dyadic normalization.[30, Section 6.1]

Every AWPP language has a GapP witness g and $p \in \mathbb{N}[X]$ such that, writing $D(x) = 2^{p(|x|)}$, one has $0 \le g(x) \le D(x)$ and

$$x \in A \Rightarrow 7D(x) \le 8g(x), \qquad x \notin A \Rightarrow 8g(x) \le D(x).$$

Three fixed rounds of the majority-of-three polynomial give this error bound; the denominator exponent is multiplied by 27. This is a fixed-size instance of Fenner's amplification method.[28]

AWPP is closed under intersection, union, difference and symmetric difference of two languages over the same encoding, as well as complement.

For SBP, every fixed integer $m > 0$ admits a #P witness f and a positive binary-output polynomial-time natural threshold k with

$$x \in A \Rightarrow k(x) \le f(x), \qquad x \notin A \Rightarrow 2^m f(x) < k(x).$$

This uses fixed powers as in Watson's gap strengthening; m is fixed independently of the input.[90, Section 1.1] Also $A \in \mathsf{SBP}$ and $B \in \mathsf{BPP}$ imply $A \cap B \in \mathsf{SBP}$, using the existing input-dependent BPP error reduction and a dyadic bound on #P counts. This does not establish intersection closure for two SBP languages. All statements use verifier-based #P and GapP; their nondeterministic-machine characterizations and uniform input-dependent AWPP amplification remain open formalization work.

`Cslib.Complexity.WPP_subset_AWPP`
`Cslib.Complexity.LWPP_subset_AWPP`
`Cslib.Complexity.exists_AWPP_gap_eighth`
`Cslib.Complexity.inter_mem_AWPP`
`Cslib.Complexity.union_mem_AWPP`
`Cslib.Complexity.diff_mem_AWPP`
`Cslib.Complexity.symmDiff_mem_AWPP`
`Cslib.Complexity.exists_SBP_gap_pow`
`Cslib.Complexity.inter_mem_SBP_of_mem_BPP`

5 Cryptographic constructions

The one-way functions and pseudorandom objects below use the library's bit-string security experiments and machine-certified efficiency predicates. Security is asymptotic against uniform strict polynomial-time probabilistic adversaries. In the one-wayness game, x is uniform among n-bit strings; the inverter receives unary n and $f(x)$ and succeeds with any preimage, even one of a different length. The function need not preserve length.

5.1 Goldreich–Levin hard-core predicate

For every one-way function f, the inner product $\langle x, r\rangle$ over $\mathbb{F}_2$ is a hard-core predicate for the padded map $(x, r) \mapsto (f(x), r)$. The formal construction splits a seed into equal halves and retains an optional spare bit, covering every seed length. The predicate is polynomial-time computable; every uniform PPT predictor has negligible advantage over a fair guess. [35]

`Cslib.Crypto.GoldreichLevin.padded_hardCore`

5.2 Pseudorandom generators from arbitrary one-way functions

Every uniform one-way function implies a deterministic polynomial-time pseudorandom generator expanding each n-bit seed to $n + 1$ bits, secure against uniform strict PPT distinguishers. The proof follows Holenstein's pseudoentropy route to the Håstad–Impagliazzo–Levin–Luby theorem. OWF existence is the sole cryptographic assumption: no entropy estimate or nonuniform advice is supplied as an extra hypothesis. The result does not claim optimized construction exponents.[42, 45]

`Cslib.Crypto.OneWay.exists_pseudorandomGenerator`

5.3 Polynomial stretch with quantitative resource bounds

Let G be a one-bit-stretch PRG and let $k : \mathbb{N} \to \mathbb{N}$ be positive and polynomial-time computable with unary input and output. Repeatedly expanding the retained n-bit seed yields a PRG with output length $n + k(n)$, secure against the same uniform strict PPT distinguishers. In particular, one-bit stretch suffices for any strictly larger, unary-polynomial-time computable target length. [9]

The construction also has a separate machine-level resource certificate. Suppose G has time/workspace bounds t_G, s_G, the count computation has bounds t_k, s_k, and every round's seed u_i satisfies $s_G(u_i) \le Q(u)$ for initial seed u. Writing $n = |u|$ and $k = k(n)$, the certified bounds are

$$\begin{aligned} \text{time} &= O\left(t_k(n) + n + k + \sum_{i<k}(t_G(u_i) + n + 1) + 1\right), \\ \text{workspace} &= O(s_k(n) + Q(u) + n + k + 1)\,. \end{aligned}$$

The exact one-bit output-length invariant is required. The workspace bound includes the final reversal buffer; a small streaming state alone does not give an output-length-independent bound for the full construction.

`Cslib.Crypto.PRGStretch.pseudorandomGenerator`
`Cslib.Crypto.PRGStretch.computableTidilyInTimeAndSpace_stretch`

5.4 GGM and adaptive pseudorandom functions

Every uniform pseudorandom generator gives an efficiently evaluable pseudorandom function family by polynomial stretching and the GGM tree construction. Security holds against uniform adaptive strict polynomial-time oracle adversaries, not just fixed query lists. At parameter n, keys, valid queries and answers have n bits; both games answer malformed queries with the empty word. Adversaries receive the unary parameter and an empty auxiliary word. Consequently every uniform one-way function implies such a PRF. These are conditional security implications, not assertions that one-way functions exist.[36, 42]

`Cslib.Crypto.PseudorandomGenerator.exists_pseudorandomFunction`
`Cslib.Crypto.OneWay.exists_pseudorandomFunction`

5.5 Generators imply one-way functions

A secure PRG G whose output length $\ell(n)$ satisfies $\ell(n) \geq 2n$ for every seed length n is itself one-way, including against inverters that return preimages of different lengths. More generally, G is one-way if ℓ is injective and $n \mapsto 2^{-(\ell(n)-n)}$ is negligible. An arbitrary PRG with positive stretch yields a one-way function after truncation and stretch amplification; the claim in this case is existence of a possibly different function. Together with HILL, this gives

$$\text{a one-way function exists} \iff \text{a pseudorandom generator exists.}$$

All security notions are the uniform strict-PPT notions specified above. Neither side's unconditional existence is asserted.[45, 54]

`Cslib.Crypto.PseudorandomGenerator.oneWay`
`Cslib.Crypto.PseudorandomGenerator.oneWay_of_injective`
`Cslib.Crypto.exists_oneWay_iff_exists_pseudorandomGenerator`

6 Circuit depth, lower bounds and barriers

Circuit classes in this section are nonuniform. Unless a different cost is specified, native De Morgan circuits charge constants, NOT, AND and OR gates, with wiring and fanout free. Formula leaf count and formula depth are separate measures; unbounded fan-in bases are identified in their entries.

6.1 Shannon–Lupanov bounds and the MCSP window

Count all NOT, AND, OR and constant gates. For every $\varepsilon > 0$, all sufficiently large n, and every n-input truth table, MCSP answers yes at any integer threshold $s \geq (1+\varepsilon)2^n/n$. For every sufficiently large n, some truth table needs more than $\lfloor 2^n/n \rfloor$ gates. A finite counting bound makes the scarcity of small circuits explicit: for $n \geq 16$, at most $2^{\lfloor 2^n/4 \rfloor}$ of the 2^{2^n} truth tables have a circuit of size at most $\lfloor 2^n/(5n) \rfloor$. These are synthesis and counting statements; they do not establish the computational hardness of deciding MCSP.[63, 80]

`Cslib.MCSP.eventually_mem`
`Cslib.MCSP.eventually_exists_not_mem`
`Cslib.MCSP.ncard_setOf_mem_le`

6.2 Barrington's theorem

Nonuniform NC^1 is exactly the class of languages recognized by polynomial-length permutation branching programs of width five. An equivalent formulation uses programs over the alternating group A_5. This is a class equality with the library's nonuniform circuit families; it does not assert a uniform-family characterization.[10, 11]

`Cslib.Circuits.Boolean.NC_one_eq_polyLength_perm_five`
`Cslib.Circuits.Boolean.NC_one_eq_polyLength_alternatingGroup`

6.3 Spira's formula balancing theorem

Every De Morgan formula with $L \geq 1$ input leaves has an equivalent formula with at most $4\lceil \log_2 L \rceil$ AND/OR gates on each input-to-output path. Constants contribute zero leaves; when $L = 0$, the formal bound is zero. Negations have zero cost in this depth measure. The theorem certifies the depth of the balanced formula, not a machine runtime for constructing it.[84]

`Cslib.Circuits.Boolean.exists_formula_depthWith_le_four_mul_clog`

6.4 Khrapchenko's bound and exact parity formula size

Let a De Morgan formula be true on A and false on B, and let $E(A,B)$ be the pairs in $A \times B$ at Hamming distance one. Its input-leaf count L (constants contribute zero) satisfies

$$|E(A,B)|^2 \le |A|\,|B|\,L.$$

In particular, parity on n bits requires at least n^2 leaves; for $n = 2^k$ this is attained. Yablonskii's construction gives, for $n = 2^k + r$ with $0 \le r \le 2^k$, a parity formula with at most $2^k(2^k + 3r)$ leaves; for every n there is one with $8L \le 9n^2$. These are formula leaf bounds, not bounds on circuits with shared subcomputations. [49, 55, 56, 92]

`Cslib.Circuits.Formula.sq_card_hammingEdges_le`
`Cslib.Circuits.Boolean.isLeast_leafSize_parity_two_pow`
`Cslib.Circuits.Boolean.exists_formula_parity_two_pow_add`
`Cslib.Circuits.Boolean.exists_formula_parity_eight_mul_leafSize_le`

6.5 Parity lower bounds by random restrictions

A depth-d circuit of unbounded fan-in AND/OR gates and negations computing parity on n bits has at least 2^s gates whenever

$$s \ge 2, \qquad s(16s+1)^d \le n.$$

Thus the finite theorem gives an exponential lower bound at every fixed depth. Depth counts negations and size counts every gate. The exponent obtained is of order $n^{1/(d+1)}$, weaker than the sharp classical Håstad exponent $n^{1/(d-1)}$.[4, 32, 40]

`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity`

6.6 Razborov–Smolensky lower bounds

For every odd prime p, parity is not in nonuniform $\mathrm{AC}^0[p]$: adding unbounded fan-in MOD_p gates to constant-depth polynomial-size AND/OR/NOT circuits does not suffice. Quantitatively, a depth-d circuit computing parity has size at least $2^{\ell+1}/5$ whenever $\ell \ge 1$ and

$$100\big(((p-1)\ell)^d + 1\big)^2 \le n + 1.$$

The prime and depth are fixed in the class separation.[74, 83]

`Cslib.Circuits.Boolean.Smolensky.two_mul_two_pow_le_size_of_computes_parity`
`Cslib.Circuits.Boolean.Smolensky.parity_notMem_AC0Mod`

6.7 Razborov's monotone clique bound

Let $p, w \ge 2$, $w+1 \le k \le n$, $p \le n$, and $2w^2 \le k-1$, and put

$$B = (p-1)^w w!, \quad A = B^2\binom{n-w-1}{k-w-1}, \quad N = (2B+B^2)(w^2)^p(k-1)^{n-p}.$$

Every monotone AND/OR circuit for k-clique on n vertices has more than s gates if $As < \binom{n}{k}$ and $2Ns < (k-1)^n$. This states the formalized finite approximation-method bound with its parameters; no unrestricted Boolean-circuit lower bound follows from it.[73]

`Cslib.Circuits.Monotone.lt_size_of_computes_clique`

6.8 The natural-proofs barrier

Assume a family of length-doubling generators of polynomial circuit size and a fixed integer $d \geq 1$ such that, for all sufficiently large seed lengths k and every m with $m^d \leq k$, no circuit of size at most 2^m distinguishes its output from uniform with advantage greater than 2^{-m}. Then no P/poly-natural property is useful against P/poly. Here natural means containing a property of density at least $2^{-O(n)}$ whose truth-table language has circuits polynomial in the table length 2^n; useful means eventually excluding the slices of every P/poly language. The security assumption is stronger than ordinary polynomial-time PRG security. [36, 75]

`Cslib.Circuits.Boolean.not_isUseful_PPoly_of_isNatural_of_generator`

6.9 Exponentially small correlation with quadratic forms

There is a nonconstructively chosen family of bilinear Boolean functions h_n with the following property. Put

$$c_* = 1 + \frac{\pi}{6 \arccos((1+2\sqrt{2})/4)} \approx 2.78125.$$

For every real $c < c_*$, there is $\gamma > 0$ such that for all sufficiently large n, every Boolean circuit of fan-in at most two, over any Boolean basis, with at most cn positive-arity gates has

$$\left| 2 \Pr_{x \in \{0,1\}^n} [C(x) = h_n(x)] - 1 \right| \leq 2^{-\gamma n}.$$

The input is uniform, constants are not charged, and the cutoff and rate are independent of the circuit. The forms are obtained by choosing submatrix-robust matrices; no efficient construction or uniform complexity-class membership is proved. The exact coefficient above is the library's frontier-method result; related correlation-bound sources are credited separately.[21, 38, 48]

`Cslib.Circuits.Frontier.exists_eventually_abs_correlation_hardForm_le_of_lt`

6.10 Weighted-threshold activity and capacity bounds

Consider real-weight threshold gates, and count as input-reading a gate with a nonzero effective coefficient on an original input; repeated wires are aggregated before testing this condition. If a circuit computes inner product modulo two on two m-bit blocks, with its output taken from a gate, let q be its number of input-reading gates and e the maximum number of those gates outputting true on one input. Then

$$2^m \leq \left(\sum_{j=0}^{e} \binom{q}{j} \right)^2.$$

The same inequality holds with q counting all gates and e their maximum simultaneous activity.[1]

`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_sq_of_circuit_hadamardCode`
`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_gateActivity_sq_of_circuit_hadamardCode`

A separate capacity bound applies to any fixed Boolean readout of a threshold program's gate values computing f on n bits. Suppose that, across every partition of the input coordinates, every rectangle contained in either fiber of f has a side with fewer than K assignments. If q gates read original inputs in the effective sense above, then

$$2^n < 4K^2 \, 4^q.$$

This statement requires the rectangle hypothesis on both fibers; it is not an unconditional lower bound for every Boolean function.[76]

`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_lt_mul_four_pow_inputGates`

6.11 Exact parity formula depth at powers of two

Let $d(f)$ be minimum De Morgan formula depth, counting binary AND/OR gates and making negations free. For $n \geq 1$,

$$\lceil \log_2(n^2) \rceil \leq d(\mathrm{PARITY}_n) \leq 2\lceil \log_2 n \rceil, \qquad d(\mathrm{PARITY}_{2^k}) = 2k.$$

The lower bound comes from formula leaf count, and the upper bound from the parity construction. This does not assert the exact minimum for every non-power-of-two arity.[49, 56, 92]

`Cslib.Circuits.Boolean.clog_sq_le_formulaDepth_parity`
`Cslib.Circuits.Boolean.formulaDepth_parity_le_two_mul_clog`
`Cslib.Circuits.Boolean.formulaDepth_parity_two_pow`

7 Proof complexity

7.1 Cook–Reckhow characterization

NP equals coNP for every encoded instance type if and only if UNSAT has a polynomially bounded Cook–Reckhow proof system: a polynomial-time function from bit strings onto unsatisfiable CNF formulas, with a preimage of polynomial length for every such formula. Polynomial boundedness asserts that short proofs exist; it does not give a polynomial-time procedure for finding them. [24]

`Cslib.Complexity.ProofSystem.NP_eq_coNP_iff_exists_polynomiallyBounded`

7.2 Resolution size–width tradeoff

If a CNF formula has initial clause width at most k, at most N variables, and a resolution refutation with at most s lines, it has a refutation of some integer width w satisfying

$$w \leq k + 2\sqrt{2N \ln s} + 2.$$

Thus a large minimum refutation width forces a large refutation size. The statement concerns proof certificates, not the runtime of a resolution or DPLL solver.[12]

`Cslib.Logic.PL.CNF.exists_refutableInWidth_le_of_refutableInSize`

8 Communication and formulas

8.1 Karchmer–Wigderson correspondence

For a Boolean function on $n > 0$ bits, minimum De Morgan formula depth, counting AND/OR gates and making negations free, equals deterministic communication complexity of the following relation: Alice receives a true input, Bob a false input, and they must output a coordinate on which the inputs differ. The equality uses the library's worst-case protocol depth and extended-natural complexity. [51]

`Cslib.Circuits.Boolean.formulaDepth_eq_ecomplexity`

8.2 Yao's minimax principle

For finite input spaces and a nonempty output space, the public-coin communication complexity of a relation with worst-case error at most ε equals the supremum, over input distributions, of the minimum deterministic communication cost with distributional error at most ε. Both directions are proved, including the minimax step; the result is not only the easy averaging lower bound. Costs are worst-case numbers of communicated bits.[58, 93]

`Cslib.Communication.PublicCoin.ecomplexity_eq_iSup_distributionalComplexity`

8.3 Newman's theorem

If each party's input space has cardinality at most 2^n, then for every $\delta > 0$, private-coin communication complexity at error $\varepsilon + \delta$ is at most public-coin complexity at error ε plus

$$\left\lceil \log_2 \left\lceil n/\delta^2 \right\rceil \right\rceil$$

bits. The formal expression uses natural ceiling and `Nat.clog`, including its zero-argument convention. The shared finite sample of protocols is chosen existentially, without a certified procedure to construct it.[65]

`Cslib.Communication.PrivateCoin.ecomplexity_le_add_clog`

8.4 Finite-set equality and inner-product rank

Equality and inequality on a finite set X with $|X| \geq 2$ each have deterministic communication complexity exactly $\lceil \log_2 |X| \rceil + 1$. The protocol model labels leaves by the answer, so the transcript determines the output; the bound includes the bit announcing the comparison result.

For inner product modulo two on two n-bit inputs, the sign matrix has rank 2^n and the 0/1 communication matrix has rank $2^n - 1$ over any field of characteristic different from two. A deterministic protocol computing it has at least $2^n - 1$ leaves labelled true. These are exact-protocol and algebraic bounds, not randomized communication lower bounds.[58]

`Cslib.Communication.ecomplexityFun_equality`
`Cslib.Communication.ecomplexityFun_not_equality`
`Cslib.Communication.rank_signMatrix_dotProduct`
`Cslib.Communication.rank_communicationMatrix_dotProduct`
`Cslib.Communication.two_pow_sub_one_le_card_true_leaves_of_run_eq_dotProduct`

9 Boolean analysis and expander walks

9.1 Hypercontractivity on the Boolean cube

For a real-valued function on a finite Boolean cube with uniform probability measure, $1 \leq p \leq 2 \leq q < \infty$, and $0 \leq \rho \leq \sqrt{(p-1)/(q-1)}$, the noise operator satisfies

$$\|T_\rho f\|_q \leq \|f\|_p.$$

This includes the $(2, 4)$ inequality at $\rho = 1/\sqrt{3}$. The indexed theorem covers the range $p \leq 2 \leq q$ and finite real exponents; it is not a claim of all hypercontractive parameter regimes.[16, 69]

`Cslib.BitString.lpNorm_noiseOp_le_lpNorm`

9.2 Expander-walk hitting bound

Let a finite undirected regular graph be represented by a rotation map with nonempty vertex and port sets, and let λ be its nonconstant spectral operator-norm bound. Start a walk at a uniform vertex and choose each port uniformly. If each set B_i has density at most $0 \leq \mu \leq 1$, then

$$\Pr[v_i \in B_i \text{ for every } 0 \leq i \leq t] \leq \mu\big(\mu + \lambda(1-\mu)\big)^t.$$

The sets may vary with time. This is a finite probability bound; machine-level error reduction and extractor evaluation costs are separate obligations. [3, 50, 87]

`Cslib.RotationMap.dens_filter_forall_walkAt_mem_le`

9.3 The Nisan–Szegedy bound on relevant coordinates

If a Boolean function on a finite cube, read as the sign $(-1)^{f(x)}$, has no nonzero Fourier coefficients on sets of more than d coordinates, then at most $d\,2^{d-1}$ coordinates are relevant: flipping an irrelevant coordinate never changes the value. For $d = 0$ the function has no relevant coordinates; the formal natural exponent uses truncated subtraction. The Fourier coefficients use the uniform measure on the cube. [67, 69]

`Cslib.BitString.card_filter_exists_update_ne_le`

9.4 Fourier bounds for decision trees

Suppose a deterministic decision tree has structural depth d and L leaves. For any real-valued readout of its output bounded in absolute value by one, the sum of absolute Fourier coefficients is at most $L \le 2^d$. For a Boolean readout, written as signs, each coefficient is an integer multiple of $2^{-\max(d-1,0)}$, and at most $L\,2^{\max(d-1,0)}$ coefficients are nonzero. Repeated queries and unreachable branches are allowed; structural size only weakens these upper bounds. This is a spectral statement, not a formalized learning algorithm.[57, 69]

`Cslib.DecisionTree.sum_abs_fourierCoeff_le_leafCount`
`Cslib.DecisionTree.exists_fourierCoeff_signVal_eq_intCast_div_two_pow_depth_sub_one`
`Cslib.DecisionTree.card_filter_fourierCoeff_signVal_ne_zero_le`

9.5 A degree-dependent obstruction to perfect expansion

For an undirected D-regular multigraph on $N \ge 2$ vertices, with $D \ge 1$, let λ be the norm of its normalized walk operator on mean-zero functions. Then

$$\frac{N - D}{D(N-1)} \le \lambda^2.$$

In particular $\lambda \ge 1/\sqrt{2D}$ if $N \ge 2D$. The rotation-map model permits loops and parallel edges. The complete graph with one loop at each vertex has $D = N$ and $\lambda = 0$, attaining the first bound. This finite trace bound is not the asymptotic Alon–Boppana theorem. [43]

`Cslib.RotationMap.div_le_sq_spectralExpansion`
`Cslib.RotationMap.one_div_sqrt_two_mul_le_spectralExpansion`
`Cslib.RotationMap.spectralExpansion_complete`

10 Sensitivity and query complexity

For total Boolean functions on n bits, write s for maximum sensitivity, bs for maximum block sensitivity, C for certificate complexity, and D for deterministic decision-tree depth. A subscript b restricts a maximum to inputs with output b; it is zero if there are no such inputs.

10.1 Polynomial relations between query measures

For every total Boolean function f,

$$C(f) \le s(f)\,bs(f) \le bs(f)^2, \qquad D(f) \le C(f)\,bs(f) \le bs(f)^3.$$

The formal development also gives $C(f) \le 2s(f)^5$ and $D(f) \le 4s(f)^9$, using the existing sensitivity theorem and degree bounds. These are upper bounds for exact deterministic queries, with no promise on the input.[18, 44, 66, 67]

`Cslib.QueryComplexity.certificateComplexity_le_sensitivity_mul_blockSensitivity`
`Cslib.QueryComplexity.certificateComplexity_le_blockSensitivity_sq`
`Cslib.DecisionTree.complexity_le_certificateComplexity_mul_blockSensitivity`
`Cslib.DecisionTree.complexity_le_blockSensitivity_pow_three`
`Cslib.QueryComplexity.certificateComplexity_le_two_mul_sensitivity_pow_five`
`Cslib.DecisionTree.complexity_le_four_mul_sensitivity_pow_nine`

10.2 Monotone measures and symmetric evasiveness

For monotone f, $C(f) = bs(f) = s(f)$ and $D(f) \le s_0(f)s_1(f) \le s(f)^2$. Separately, every nonconstant symmetric Boolean function on n bits is evasive: $D(f) = n$. Symmetry means dependence only on Hamming weight. Thus, for Th_j^n, which accepts iff at least j bits are true, with $1 \le j \le n$,

$$D(\mathrm{Th}_j^n) = n, \qquad C(\mathrm{Th}_j^n) = bs(\mathrm{Th}_j^n) = s(\mathrm{Th}_j^n) = \max(j, n+1-j).$$

The equality of the three local-change and certificate measures requires monotonicity; it is not asserted for arbitrary functions. [18, 66]

`Cslib.QueryComplexity.certificateComplexity_eq_sensitivity_of_monotone`
`Cslib.QueryComplexity.blockSensitivity_eq_sensitivity_of_monotone`
`Cslib.DecisionTree.complexity_le_sensitivityFor_mul_sensitivityFor_of_monotone`
`Cslib.DecisionTree.complexity_eq_card_of_isSymmetric`
`Cslib.BitString.sensitivity_threshold`

10.3 Approximate degree of disjunction

Let $\widetilde{\deg}_{1/3}(f)$ be the least degree of a real polynomial approximating the 0/1 value of f to error at most $1/3$ at every Boolean input. For disjunction on n bits,

$$n \le 6\,\widetilde{\deg}_{1/3}(\mathrm{OR}_n)^2, \qquad \widetilde{\deg}_{1/3}(\mathrm{OR}_n) \le \lfloor \sqrt{2n} \rfloor + 1.$$

These explicit finite bounds establish the square-root order of growth. The upper bound uses a Chebyshev polynomial; no optimal leading constant is claimed. [18, 67]

`Cslib.QueryComplexity.card_le_six_mul_approxDegree_threshold_one_right_sq`
`Cslib.QueryComplexity.approxDegree_threshold_one_right_le_sqrt`

10.4 Superquadratic separation of sensitivity and block sensitivity

There is a fixed real exponent $\alpha > 2$ and a family of nonconstant Boolean functions F_m on positive, varying arities such that $F_m(0) = 0$ and

$$s(F_m)^\alpha \le bs(F_m, 0), \qquad bs(F_m, 0) \longrightarrow \infty.$$

Here 0 is the all-false input, and $bs(f, 0)$ is block sensitivity at that specific input. In particular, for every real c there exists a nonconstant f with $c\,s(f)^2 < bs(f)$, so no universal quadratic upper bound holds. The exponent is existential, not an assertion for every $\alpha > 2$. The linked construction ports OpenAI's formalization through the earlier complexitylib development and connects it to the library's public measures. [70, 79]

`Cslib.QueryComplexity.exists_sensitivity_rpow_le_blockSensitivityAt`
`Cslib.QueryComplexity.exists_mul_sensitivity_sq_lt_blockSensitivity`

11 Finite combinatorial inequalities

11.1 Discrete Loomis–Whitney

For a finite set A in a product of $n \ge 2$ finite coordinate spaces, let $\pi_i(A)$ be its projection obtained by deleting coordinate i. Then

$$|A|^{n-1} \le \prod_{i=1}^{n} |\pi_i(A)|.$$

The coordinate spaces may differ and A may be empty. This is the discrete cardinality inequality, obtained from the counting form of Shearer's lemma; no continuous-volume theorem is asserted here. [22, 62]

`Cslib.Probability.PMF.card_pow_le_prod_card_image_restrict_erase`

11.2 A lower bound for sunflower thresholds

For every $p \geq 2$ and $w \geq 0$, there is a family of exactly $(p-1)^w$ distinct sets, each of size w, containing no sunflower with p petals. The construction uses the graphs of all functions from a w-element set to a $(p-1)$-element set. A sunflower means distinct sets with all pairwise intersections equal to a common core. This lower bound need not be optimal: the six edges of two disjoint triangles are a family of two-element sets with no three-petal sunflower. [26]

`Cslib.exists_card_eq_pow_not_containsSunflower`
`Cslib.Sunflower.card_twoTriangles`
`Cslib.Sunflower.card_of_mem_twoTriangles`
`Cslib.Sunflower.not_containsSunflower_twoTriangles`

12 Reed–Solomon decoding

12.1 Berlekamp–Welch unique decoding

Let $\alpha_1, \ldots, \alpha_n$ be distinct points of a field, and let w be a received word. If $2e + k \leq n$, the decoder returns a polynomial P exactly when $\deg P < k$ and its evaluation word differs from w in at most e positions. It returns failure exactly when no such polynomial exists. The formal decoder chooses a solution of the key equations noncomputably; its correctness is proved, but polynomial decoding runtime is not certified.[39, 91]

`Cslib.Code.berlekampWelch_eq_some_iff`
`Cslib.Code.berlekampWelch_eq_none_iff`

12.2 Sudan's list-decoding guarantee

For distinct field evaluation points, $k \geq 2$, and a natural number D satisfying

$$2(k-1)n < (D+1)(D+2),$$

Sudan's decoder lists exactly the polynomials of degree below k agreeing with the received word at more than D positions. The number of listed polynomials is at most $\lfloor D/(k-1) \rfloor$. Interpolation and root containment are proved; the interpolant is chosen noncomputably, and no decoding runtime or Guruswami–Sudan multiplicity improvement is asserted.[39, 85]

`Cslib.Code.mem_sudan_iff_of_lt`
`Cslib.Code.card_sudan_mul_le`

13 Arithmetic circuits and simultaneous computation

13.1 Polynomial-size circuits for the determinant

For every commutative ring K and $n \in \mathbb{N}$, the generic $n \times n$ determinant has a division-free arithmetic circuit over K of size at most

$$3n^4 + 4.$$

Consequently its polynomial family belongs to VP over K. The same circuit evaluates determinants over every commutative K-algebra, with matrix entries supplied in row-major order. This is a gate-count assertion, not a certified uniform circuit-generation bound.[13, 89]

`Cslib.Circuits.Arithmetic.exists_circuit_det`
`Cslib.Circuits.Arithmetic.exists_circuit_det_eval`
`Cslib.Circuits.Arithmetic.detFamily_mem_VP`

13.2 Sharp circuit mass production

Fix real $0 \le \gamma < 1$ and $\varepsilon > 0$. For every sufficiently large n, every Boolean function f on n bits, and every integer $1 \le k \le 2^{\gamma n}$, one circuit computes f independently on k disjoint n-bit input blocks with nonconstant-gate cost at most

$$\left(\frac{1}{1-\gamma} + \varepsilon\right)\frac{2^n}{n}.$$

The cutoff depends on γ, ε, not on f, k. Here NOT, AND and OR each cost one and constants cost zero, unlike the total-gate convention used in other entries. The construction is nonuniform and supplies no certified runtime for choosing its tables.[78]

`Cslib.Circuits.Boolean.MassProduction.realSharpMassProduction`

14 Circuits for iterated functions

14.1 Fixed powers of arbitrary finite-state maps

Given a native De Morgan circuit of size s computing a map f on n-bit states, every fixed natural exponent T and integer $1 \le L \le 2^n$ admit a circuit for f^T of size at most

$$40000\left(Ls + (n+1)^3\left(L + \left\lfloor\frac{2^n}{L}\right\rfloor + 1\right)\right).$$

The map need not be invertible. The bound is independent of T, but T is fixed when the circuit is chosen; it is not an input to this circuit. Checkpoints are chosen nonconstructively, without a certified preprocessing runtime. [77]

`Cslib.Circuits.Boolean.Iteration.fixed_power`

14.2 An input time for an arbitrary finite-state map

With f, s, n, L as in the fixed-power result, define

$$B = Ls + (n+1)^3\left(L + \left\lfloor\frac{2^n}{L}\right\rfloor + 1\right).$$

For every $m \in \mathbb{N}$, one circuit on $n+m$ inputs computes $f^T(x)$ for all n-bit states x and all m-bit times $0 \le T < 2^m$, using at most

$$m(40000B + 4n)$$

gates. The time is supplied least-significant bit first. This theorem allows noninvertible maps; its linear dependence on the time width differs from the additive sparse-counter bound below. Circuit existence is nonuniform, with no certified checkpoint-construction runtime.[77]

`Cslib.Circuits.Boolean.Iteration.variable_time`

14.3 Batched iteration with a shared table

Let a size-s native De Morgan circuit compute f on n bits. For integers $0 \le r \le 2^n$ and $1 \le L \le 2^n$, put

$$B_r = rLs + (n+1)^3 \left(rL + \left\lfloor \frac{2^n}{L} \right\rfloor + 1 \right).$$

For every fixed $T \in \mathbb{N}$, one circuit applies f^T to r independent states with at most $100000B_r$ gates. With a separate m-bit input time for each state, a circuit of size at most

$$m(100000B_r + 4rn)$$

computes all r answers. The table term $\lfloor 2^n/L \rfloor$ is charged once in B_r, rather than once per state. All gates, including constants, are charged. These are nonuniform size bounds, not certified preprocessing or circuit-generation times.[77]

`Cslib.Circuits.Boolean.Iteration.batched_fixed_power`
`Cslib.Circuits.Boolean.Iteration.batched_variable_time`

Sparse-counter conventions

Write $W_n = \mathbb{Z}/2^n\mathbb{Z}$ for the n-bit words, represented least-significant bit first, and let $c(x) = x + 1$. For maps on W_n, put

$$D(f, g) = \{x \in W_n : f(x) \ne g(x)\}.$$

Circuit size below counts native De Morgan gates, including constants, comparisons, selection and addition. These are nonuniform existence results: they provide no efficient algorithm for constructing the correction table. The port follows Schlesinger's iterated-functions development; the source credits Moreno and Rivera for the related block characterization of almost-commuting permutations. [64, 77]

14.4 Maps commuting with an edited counter

For any maps $f, h : W_n \to W_n$, if h is injective and $h \circ f = f \circ h$, then there is a circuit computing all n output bits of h with size at most

$$100(n+1)(|D(f, c)| + 1).$$

There is no assumption that f is a permutation or that a small circuit for f is supplied. The assertion includes $n = 0$.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_centralizer`

14.5 Fixed signed powers and coordinate transfer

For every permutation f of W_n and every integer T, a circuit computes f^T with size at most

$$100(n+1)(|D(f, c)| + 1).$$

The size bound is independent of the magnitude and sign of T. The quantifiers are "for every T, there exists a circuit": T is fixed, not an additional input to a single circuit.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_power`

Coordinate transfer. Let f, g, e be permutations of W_n such that $e(g(x)) = e(x) + 1$ for every x. Suppose circuits E and E^{-1} compute e and its inverse, respectively. For every $T \in \mathbb{Z}$, there is a circuit for f^T with size at most

$$|E| + |E^{-1}| + 100(n+1)(|D(f,g)| + 1).$$

Both coordinate conversions are charged. The theorem does not assert that an arbitrary cycle has cheap coordinates or complete the general iterated-function circuit-bound program.
`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_power`

An input iteration count

Keep the notation W_n, c and D introduced above. Here a circuit reads an n-bit initial state x and an m-bit nonnegative integer T, both least-significant bit first. The permutation is fixed when choosing the circuit. The chart and circuit are chosen nonuniformly; no efficient preprocessing or circuit-generation algorithm is asserted.[77]

14.6 Sparse-counter input times and cycle labels

For every permutation f of W_n and every $m \in \mathbb{N}$, there is a native De Morgan circuit with $n + m$ inputs and n outputs computing

$$(x, T) \longmapsto f^T(x), \qquad 0 \le T < 2^m,$$

of size at most

$$1000(n+3)(|D(f,c)| + m + 1).$$

The dependence on the edit count and time width is additive inside the parentheses. One circuit handles all initial states and all m-bit times. The bound includes constants, comparisons and selection gates, and permits $n = 0$ or $m = 0$.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_variable_time`

Canonical coordinates. For every permutation f of W_n, there is a circuit with n inputs and $n + 2(n+2)$ outputs computing the canonical cycle representative, cycle length and phase of the input. Its size is at most

$$500(n+3)(|D(f,c)| + 1).$$

The representative is the numerically least word on the cycle. The phase is the forward distance from that representative, modulo the cycle length. The period and phase each occupy $n + 2$ bits, so even a period of 2^n fits.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_labels`

Coordinate transfer. Let f, g, e be permutations of W_n satisfying $e(g(x)) = e(x) + 1$, with supplied circuits E and E^{-1} for e and its inverse. For every time width m, a circuit computing $(x, T) \mapsto f^T(x)$ has size at most

$$|E| + |E^{-1}| + 1000(n+3)(|D(f,g)| + m + 1).$$

The m time bits pass through unchanged to the iteration stage. Both state coordinate conversions are charged once, independently of m.
`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_variable_time`

References

[1] Threshold circuits as decision trees and finite-state programs. Research draft, second pass, 2026. URL https://github.com/SamuelSchlesinger/threshold-circuits/blob/791fa2749317fbcc7b5c5b030a1e6fadd28cc44c/threshold_circuit_representations.tex. October 18, 2026; repository revision 791fa2749317fbcc7b5c5b030a1e6fadd28cc44c.

[2] Leonard Adleman. Two theorems on random polynomial time. In *19th Annual Symposium on Foundations of Computer Science*, pages 75–83. IEEE Computer Society, 1978. doi: 10.1109/SFCS.1978.37.

[3] M. Ajtai, J. Komlós, and E. Szemerédi. Deterministic simulation in LOGSPACE. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, pages 132–140. ACM, 1987. doi: 10.1145/28395.28410. URL https://doi.org/10.1145/28395.28410.

[4] Miklós Ajtai. Σ_1^1-formulae on finite structures. *Annals of Pure and Applied Logic*, 24(1):1–48, 1983. doi: 10.1016/0168-0072(83)90038-6.

[5] Eric W. Allender and Roy S. Rubinstein. P-printable sets. *SIAM Journal on Computing*, 17 (6):1193–1202, 1988. doi: 10.1137/0217075.

[6] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.

[7] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998. doi: 10.1145/278298.278306.

[8] Theodore Baker, John Gill, and Robert Solovay. Relativizations of the $\mathcal{P} =? \mathcal{NP}$ question. *SIAM Journal on Computing*, 4(4):431–442, 1975. doi: 10.1137/0204037.

[9] Boaz Barak. CS 127: Pseudorandomness, 2016. URL https://www.boazbarak.org/cs127spring16/chap03-pseudorandom-generators. Length extension for pseudorandom generators.

[10] David A. Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1. *Journal of Computer and System Sciences*, 38(1):150–164, 1989.

[11] David A. Mix Barrington and Denis Thérien. Finite monoids and the fine structure of NC^1. *Journal of the ACM*, 35(4):941–952, 1988. doi: 10.1145/48014.63138.

[12] Eli Ben-Sasson and Avi Wigderson. Short proofs are narrow—resolution made simple. *Journal of the ACM*, 48(2):149–169, 2001. doi: 10.1145/375827.375835. URL https://doi.org/10.1145/375827.375835.

[13] Richard S. Bird. A simple division-free algorithm for computing determinants. *Information Processing Letters*, 111(21–22):1072–1074, 2011. doi: 10.1016/j.ipl.2011.08.006.

[14] Andreas Blass and Yuri Gurevich. On the unique satisfiability problem. *Information and Control*, 55(1–3):80–88, 1982. doi: 10.1016/S0019-9958(82)90439-9.

[15] Elmar Böhler, Christian Glaßer, and Daniel Meister. Error-bounded probabilistic computations between MA and AM. *Journal of Computer and System Sciences*, 72(6):1043–1076, 2006.

[16] Aline Bonami. Étude des coefficients de Fourier des fonctions de $L^p(G)$. *Annales de l'Institut Fourier*, 20(2):335–402, 1970. doi: 10.5802/aif.357.

[17] Bernd Borchert, Lane A. Hemaspaandra, and Jörg Rothe. Restrictive acceptance suffices for equivalence problems. *LMS Journal of Computation and Mathematics*, 3:86–95, 2000. doi: 10.1112/S146115700000022X. URL https://arxiv.org/abs/cs/9907041.

[18] Harry Buhrman and Ronald de Wolf. Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science*, 288(1):21–43, 2002.

[19] Jin-Yi Cai, Thomas Gundermann, Juris Hartmanis, Lane A. Hemachandra, Vivian Sewelson, Klaus Wagner, and Gerd Wechsung. The boolean hierarchy I: Structural properties. *SIAM Journal on Computing*, 17(6):1232–1252, 1988. doi: 10.1137/0217078.

[20] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114–133, 1981. doi: 10.1145/322234.322243.

[21] Ruiwen Chen and Valentine Kabanets. Correlation bounds and #sat algorithms for small linear-size circuits. In *Computing and Combinatorics (COCOON 2015)*, Lecture Notes in Computer Science, pages 211–222. Springer, 2015. doi: 10.1007/978-3-319-21398-9_17.

[22] Fan R. K. Chung, Ronald L. Graham, Peter Frankl, and James B. Shearer. Some intersection theorems for ordered sets and graphs. *Journal of Combinatorial Theory, Series A*, 43(1):23–37, 1986. doi: 10.1016/0097-3165(86)90019-1.

[23] James Cook and Ian Mertz. Tree evaluation is in space $O(\log n \cdot \log \log n)$. Technical Report TR23-174, Electronic Colloquium on Computational Complexity, 2023. Conference version in STOC 2024.

[24] Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *The Journal of Symbolic Logic*, 44(1):36–50, 1979. doi: 10.2307/2273702.

[25] Irit Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3):12, 2007. doi: 10.1145/1236457.1236459.

[26] Paul Erdős and Richard Rado. Intersection theorems for systems of sets. *Journal of the London Mathematical Society*, s1-35(1):85–90, 1960. doi: 10.1112/jlms/s1-35.1.85.

[27] Ronald Fagin. Generalized first-order spectra and polynomial-time recognizable sets. In Richard M. Karp, editor, *Complexity of Computation*, volume 7 of *SIAM-AMS Proceedings*, pages 43–73. 1974.

[28] Stephen A. Fenner. PP-lowness and a simple definition of AWPP. *Theory of Computing Systems*, 36(2):199–212, 2003. doi: 10.1007/s00224-002-1089-8. URL https://eccc.weizmann.ac.il/report/2002/036/.

[29] Stephen A. Fenner, Lance J. Fortnow, and Stuart A. Kurtz. Gap-definable counting classes. *Journal of Computer and System Sciences*, 48(1):116–148, 1994. doi: 10.1016/S0022-0000(05)80024-8.

[30] Stephen A. Fenner, Lance J. Fortnow, Stuart A. Kurtz, and Lide Li. An oracle builder's toolkit. *Information and Computation*, 182(2):95–136, 2003. doi: 10.1016/S0890-5401(03)00018-X. URL https://lance.fortnow.com/papers/files/obt.pdf.

[31] Lance Fortnow. Counting complexity. In Lane A. Hemaspaandra and Alan L. Selman, editors, *Complexity Theory Retrospective II*, pages 81–107. Springer, 1997. doi: 10.1007/978-1-4612-1872-2_4. URL https://lance.fortnow.com/papers/files/counting.pdf.

[32] Merrick Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. *Mathematical Systems Theory*, 17(1):13–27, 1984. doi: 10.1007/BF01744431.

[33] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.

[34] Oded Goldreich. On the Cook–Mertz tree evaluation procedure. Technical Report TR24-109, Electronic Colloquium on Computational Complexity, 2024. Revision 2, 27 December 2024.

[35] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, pages 25–32, 1989. doi: 10.1145/73007.73010. URL https://www.wisdom.weizmann.ac.il/~oded/X/gl.pdf.

[36] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986. URL https://www.wisdom.weizmann.ac.il/~oded/X/ggm-jacm.pdf.

[37] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):690–728, 1991. doi: 10.1145/116825.116852.

[38] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Circuit size lower bounds and #sat upper bounds through a general framework. In *41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016)*, volume 58 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 45:1–45:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. doi: 10.4230/LIPIcs.MFCS.2016.45.

[39] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. Essential coding theory, 2019. URL https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/. Draft book, 2019.

[40] Johan Håstad. Almost optimal lower bounds for small depth circuits. In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing*, pages 6–20, 1986. doi: 10.1145/12130.12132.

[41] Edith Hemaspaandra, Lane A. Hemaspaandra, Holger Spakowski, and Osamu Watanabe. The robustness of LWPP and WPP, with an application to graph reconstruction. In *43rd International Symposium on Mathematical Foundations of Computer Science*, volume 117 of *Leibniz International Proceedings in Informatics*, pages 51:1–51:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.MFCS.2018.51.

[42] Thomas Holenstein. Pseudorandom generators from one-way functions: A simple construction for any hardness. In *Theory of Cryptography (TCC 2006)*, 2006. URL https://crypto.ethz.ch/publications/files/Holens06.pdf.

[43] Shlomo Hoory, Nathan Linial, and Avi Wigderson. Expander graphs and their applications. *Bulletin of the American Mathematical Society*, 43(4):439–561, 2006. doi: 10.1090/S0273-0979-06-01126-8.

[44] Hao Huang. Induced subgraphs of hypercubes and a proof of the sensitivity conjecture. *Annals of Mathematics*, 190(3):949–955, 2019. doi: 10.4007/annals.2019.190.3.6.

[45] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999. doi: 10.1137/S0097539793244708. URL https://doi.org/10.1137/S0097539793244708.

[46] Neil Immerman. *Descriptive Complexity*. Graduate Texts in Computer Science. Springer, 1999. doi: 10.1007/978-1-4612-0539-5.

[47] Neil Immerman and Susan Landau. The complexity of iterated multiplication. *Information and Computation*, 116(1):103–116, 1995. URL https://people.cs.umass.edu/~immerman/pub/mult.pdf.

[48] Stasys Jukna. *Extremal Combinatorics: With Applications in Computer Science*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2 edition, 2011. doi: 10.1007/978-3-642-17364-6.

[49] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*, volume 27 of *Algorithms and Combinatorics*. Springer, 2012. doi: 10.1007/978-3-642-24508-4. URL https://web.vu.lt/mif/s.jukna/boolean/index.html.

[50] Nabil Kahale. Eigenvalues and expansion of regular graphs. *Journal of the ACM*, 42(5): 1091–1106, 1995. doi: 10.1145/210118.210136. URL https://doi.org/10.1145/210118.210136.

[51] Mauricio Karchmer and Avi Wigderson. Monotone circuits for connectivity require super-logarithmic depth. *SIAM Journal on Discrete Mathematics*, 3(2):255–265, 1990.

[52] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[53] Richard M. Karp and Richard J. Lipton. Some connections between nonuniform and uniform complexity classes. In *Proceedings of the 12th Annual ACM Symposium on Theory of Computing*, STOC 1980, pages 302–309, 1980. doi: 10.1145/800141.804678.

[54] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3rd edition, 2020. ISBN 9780815354369.

[55] V. M. Khrapchenko. Complexity of the realization of a linear function in the class of π-circuits. *Mathematical Notes of the Academy of Sciences of the USSR*, 9:21–23, 1971. Russian original: Matematicheskie Zametki 9(1), 35–40.

[56] V. M. Khrapchenko. Method of determining lower bounds for the complexity of π-schemes. *Mathematical Notes of the Academy of Sciences of the USSR*, 10:474–479, 1971. doi: 10.1007/BF01747074. Russian original: Matematicheskie Zametki 10(1), 83–92.

[57] Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the Fourier spectrum. *SIAM Journal on Computing*, 22(6):1331–1348, 1993. doi: 10.1137/0222080.

[58] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[59] Richard E. Ladner. The circuit value problem is log space complete for P. *ACM SIGACT News*, 7(1):18–20, 1975. doi: 10.1145/990518.990519.

[60] Clemens Lautemann. BPP and the polynomial hierarchy. *Information Processing Letters*, 17 (4):215–217, 1983. doi: 10.1016/0020-0190(83)90044-3.

[61] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Springer, Cham, 4th edition, 2019. doi: 10.1007/978-3-030-11298-1.

[62] Lynn H. Loomis and Hassler Whitney. An inequality related to the isoperimetric inequality. *Bulletin of the American Mathematical Society*, 55(10):961–962, 1949. doi: 10.1090/S0002-9904-1949-09320-5.

[63] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya Vysshikh Uchebnykh Zavedenii. Radiofizika*, 1(1):120–140, 1958. URL https://radiophysics.unn.ru/sites/default/files/papers/1958_1_120.pdf. In Russian.

[64] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5:1949, 2016. doi: 10.1186/s40064-016-3638-7. URL https://doi.org/10.1186/s40064-016-3638-7. Article 1949.

[65] Ilan Newman. Private vs. common random bits in communication complexity. *Information Processing Letters*, 39(2):67–71, 1991. doi: 10.1016/0020-0190(91)90157-D.

[66] Noam Nisan. CREW PRAMs and decision trees. *SIAM Journal on Computing*, 20(6): 999–1007, 1991.

[67] Noam Nisan and Mario Szegedy. On the degree of Boolean functions as real polynomials. *Computational Complexity*, 4(4):301–313, 1994. doi: 10.1007/BF01263419.

[68] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994. doi: 10.1016/S0022-0000(05)80043-1.

[69] Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. doi: 10.1017/CBO9781139814782.

[70] OpenAI. Sensitivity versus block sensitivity: a superquadratic separation, 2026. URL https://github.com/openai/math/tree/adc7f1241b42e322a6451854ab7e4b4c146bf78a/lean/OAI/Combinatorics/Sensitivity. Lean formalization, revision adc7f1241b42e322a6451854ab7e4b4c146bf78a.

[71] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, Massachusetts, 1994.

[72] Christos H. Papadimitriou and Stathis K. Zachos. Two remarks on the power of counting. In Armin B. Cremers and Hans-Peter Kriegel, editors, *Theoretical Computer Science: 6th GI-Conference, Dortmund, January 5–7, 1983*, volume 145 of *Lecture Notes in Computer Science*, pages 269–275. Springer-Verlag, 1982. doi: 10.1007/BFb0009651. URL https://doi.org/10.1007/BFb0009651. Proceedings published in December 1982 for the January 1983 conference.

[73] Alexander A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Soviet Mathematics Doklady*, 31:354–357, 1985. Russian original: Doklady Akademii Nauk SSSR 281(4), 798–801.

[74] Alexander A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41 (4):333–338, 1987. doi: 10.1007/BF01137685.

[75] Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55(1):24–35, 1997. doi: 10.1006/jcss.1997.1494.

[76] Samuel Schlesinger. Algebraic circuit formalizations in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/tree/2ad6cb481b70ee6e65e078dffb76b9856be199d3/Complexitylib/Algebraic. Source of ported Lean proofs, revision 2ad6cb481b70ee6e65e078dffb76b9856be199d3.

[77] Samuel Schlesinger. Square-root circuit bounds for iterated functions, 2026. URL https://github.com/SamuelSchlesinger/iterating-functions/tree/c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b/lean. Source of ported Lean proofs, revision c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b.

[78] Samuel Schlesinger. Boolean circuit mass production formalization, 2026. URL https://github.com/SamuelSchlesinger/algebraic-circuits/tree/3998dc48823f7d2798aa814f2cbfbf3ceba146fa/Algebraic. Source of ported Lean proofs, revision 3998dc48823f7d2798aa814f2cbfbf3ceba146fa.

[79] Samuel Schlesinger. Sensitivity separation in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/tree/add0aca8ee0d71e68969f2cb45cf91554a50f787/Complexitylib/BooleanAnalysis/Sensitivity. Source of ported Lean proofs, revision add0aca8ee0d71e68969f2cb45cf91554a50f787.

[80] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi: 10.1002/j.1538-7305.1949.tb03624.x. URL https://deeplearning.cs.cmu.edu/F24/document/readings/Shannon49.pdf.

[81] Michael Sipser. A complexity theoretic approach to randomness. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 330–335, 1983. doi: 10.1145/800061.808762. URL https://doi.org/10.1145/800061.808762.

[82] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.

[83] Roman Smolensky. Algebraic methods in the theory of lower bounds for Boolean circuit complexity. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, pages 77–82, 1987. doi: 10.1145/28395.28404.

[84] Philip M. Spira. On time-hardware complexity tradeoffs for Boolean functions. In *Proceedings of the Fourth Hawaii International Symposium on System Sciences*, pages 525–527, 1971.

[85] Madhu Sudan. Decoding of Reed Solomon codes beyond the error-correction bound. *Journal of Complexity*, 13(1):180–193, 1997. doi: 10.1006/jcom.1997.0439.

[86] Seinosuke Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991. doi: 10.1137/0220053. URL https://doi.org/10.1137/0220053.

[87] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1–3):1–336, 2012. doi: 10.1561/0400000010.

[88] Leslie G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979. doi: 10.1016/0304-3975(79)90044-6.

[89] Leslie G. Valiant. Completeness classes in algebra. In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, STOC '79, pages 249–261. ACM, 1979. doi: 10.1145/800135.804419.

[90] Thomas Watson. The complexity of estimating min-entropy. Technical Report TR12-070, Electronic Colloquium on Computational Complexity, 2012. URL https://eccc.weizmann.ac.il/report/2012/070/.

[91] Lloyd R. Welch and Elwyn R. Berlekamp. Error correction for algebraic block codes, 1986. U.S. Patent 4,633,470.

[92] S. V. Yablonskii. Realization of a linear function in the class of π-schemes. *Doklady Akademii Nauk SSSR*, 94(5):805–806, 1954. In Russian.

[93] Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, FOCS 1977, pages 222–227, 1977. doi: 10.1109/SFCS.1977.24.